Name: Bhavya Sahithi G
USN: 1RUA25BCA0014

STUDENT RECORD MANAGEMENT SYSTEM

Mini Project Report

Technologies: Java Servlets • JDBC • MySQL • HTML • Apache Tomcat • NetBeans

Abstract

The Student Record Management System is a browser-based application designed to manage student information through a simple web interface. The application supports adding new student records, viewing stored records, updating existing information, and deleting selected records. Java Servlets handle server-side requests, JDBC connects the application to a MySQL database, and HTML provides the user interface. The project demonstrates CRUD operations on persistent database records.

1. Introduction

Student information must be stored and maintained accurately. A database-backed web application helps reduce manual record handling and allows information to be created, retrieved, modified, and removed through a browser. This project implements the basic operations required for student record management.

2. Problem Statement

Managing student details manually can lead to repeated data entry, difficulty finding records, and errors when information changes. The objective of this project is to provide a simple web application that stores student details in a MySQL database and allows controlled operations through a webpage.

3. Objectives

- Develop a web-based student record management application.
- Connect Java Servlet code to MySQL using JDBC.
- Implement Create, Read, Update, and Delete (CRUD) operations.
- Store student records persistently in a relational database.
- Test database changes through the website and MySQL Workbench.

4. Software and Technologies Used

Technology	Purpose
HTML	Creates the web forms and page structure.
Java Servlets	Processes browser requests and performs application logic.
JDBC	Provides connectivity between Java and MySQL.
MySQL	Stores student records in the student_db database.
Apache Tomcat	Runs and serves the Java web application.
Apache NetBeans	IDE used to develop, build, and deploy the project.
MySQL Workbench	Used to execute SQL queries and verify stored records.

5. System Design and Database Structure

The browser sends a request to a Java Servlet running on Apache Tomcat. The servlet uses the DBConnection class and JDBC to communicate with the MySQL database. The result is returned to the browser as a webpage.

Browser (HTML) → Apache Tomcat / Java Servlets → JDBC → MySQL

Database: student_db
Table: students

Column	Data type	Purpose
student_id	INT, Primary Key, Auto Increment	Unique identifier for each student.
name	VARCHAR(100)	Student name; required.
email	VARCHAR(100)	Student email; required and unique.
course	VARCHAR(50)	Course name; required.
semester	INT	Current semester; required.

6. Application Modules

Add Student: Accepts student name, email, course, and semester through an HTML form and inserts the data into MySQL.

View Students: Retrieves saved records from the students table and displays them in a table on the webpage.

Update Student: Updates the selected student record using the student ID and new details.

Delete Student: Deletes the selected record using its student ID after the user submits the delete form.

Database Connection: The DBConnection class establishes a JDBC connection to the student_db database.

7. Implementation Screenshots

The following screenshots document the application interface, record operations, database verification, and build/deployment environment.



Figure 1. Homepage showing the four student record operations.

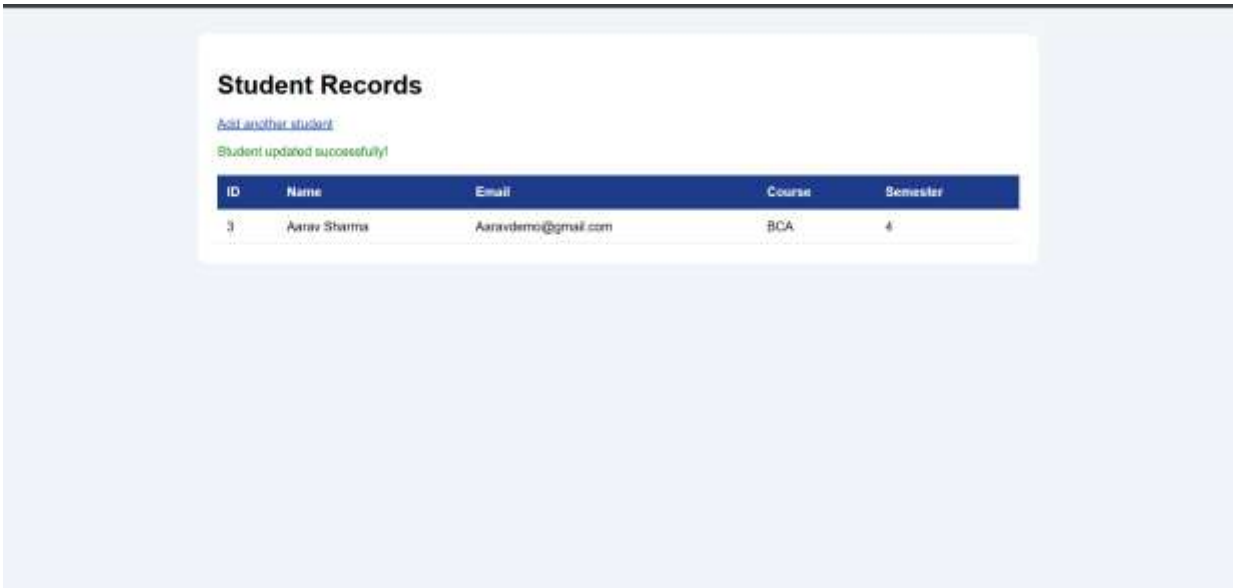


Figure 2. View Students page showing a saved student record.

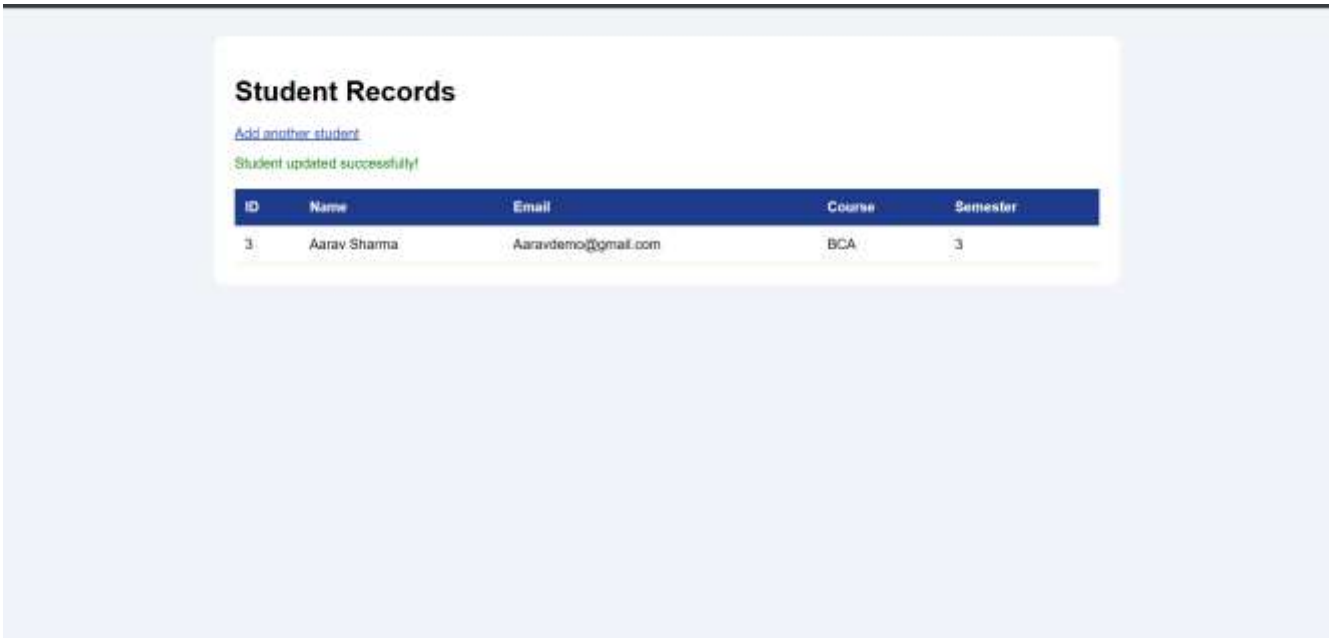


Figure 3. Update operation with the success message and updated semester.

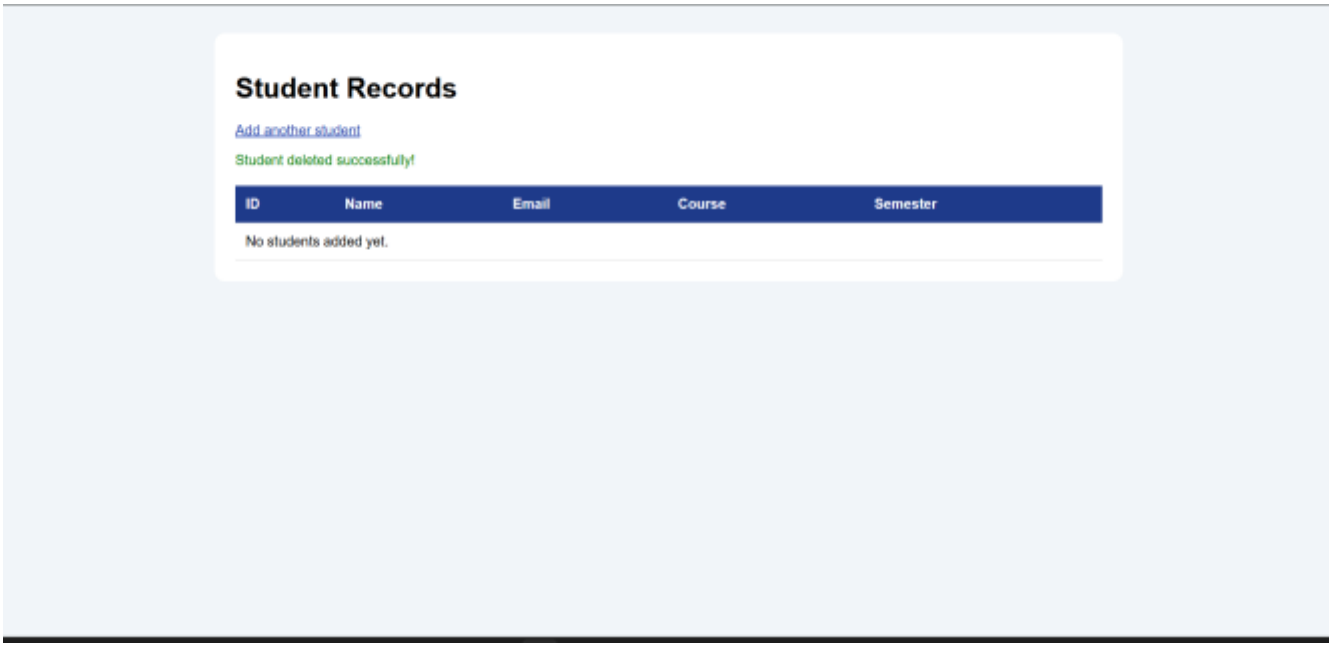


Figure 4. Delete operation with confirmation message and the resulting empty list.

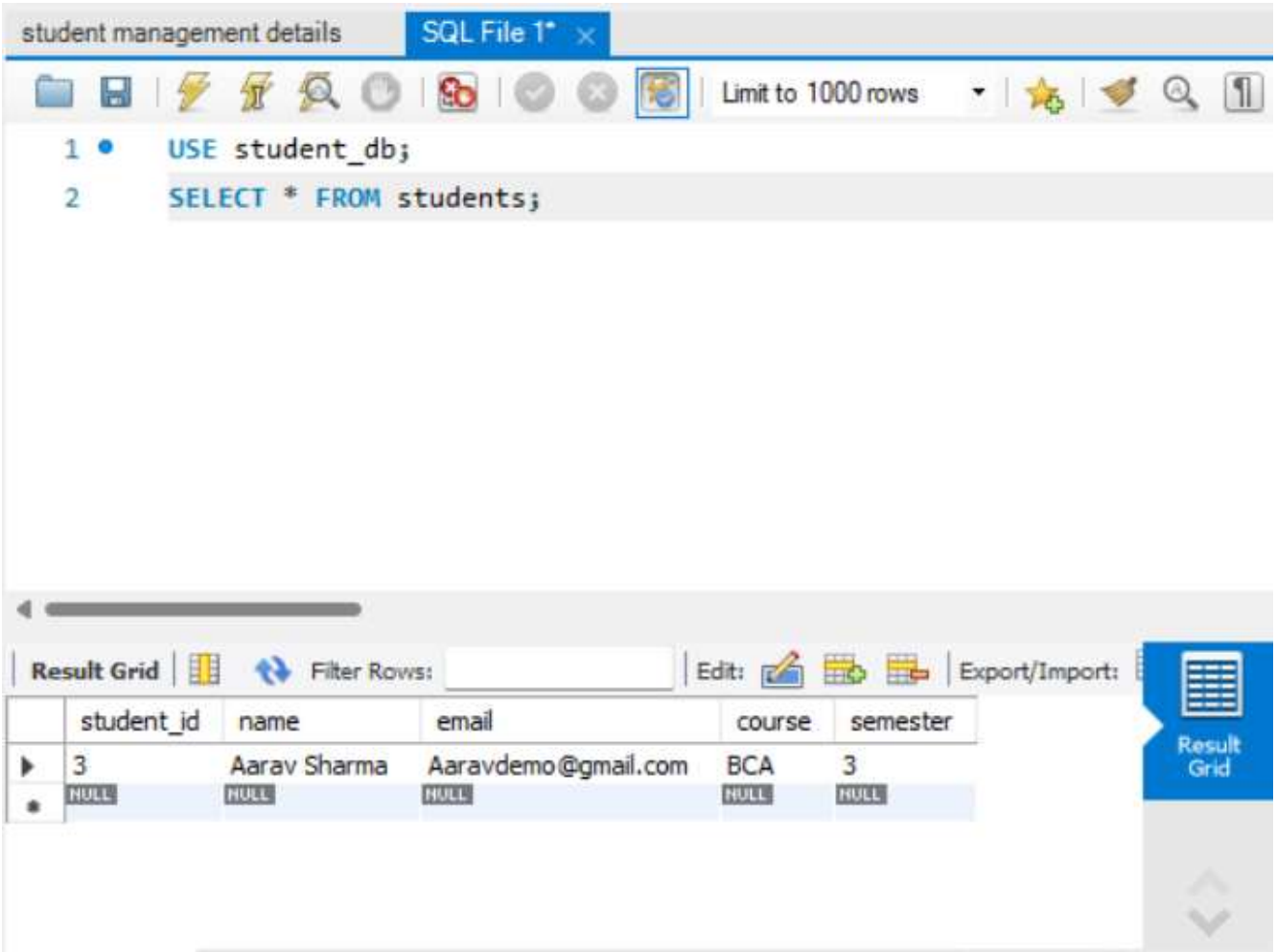


Figure 5. MySQL Workbench result grid showing the student record in student_db.

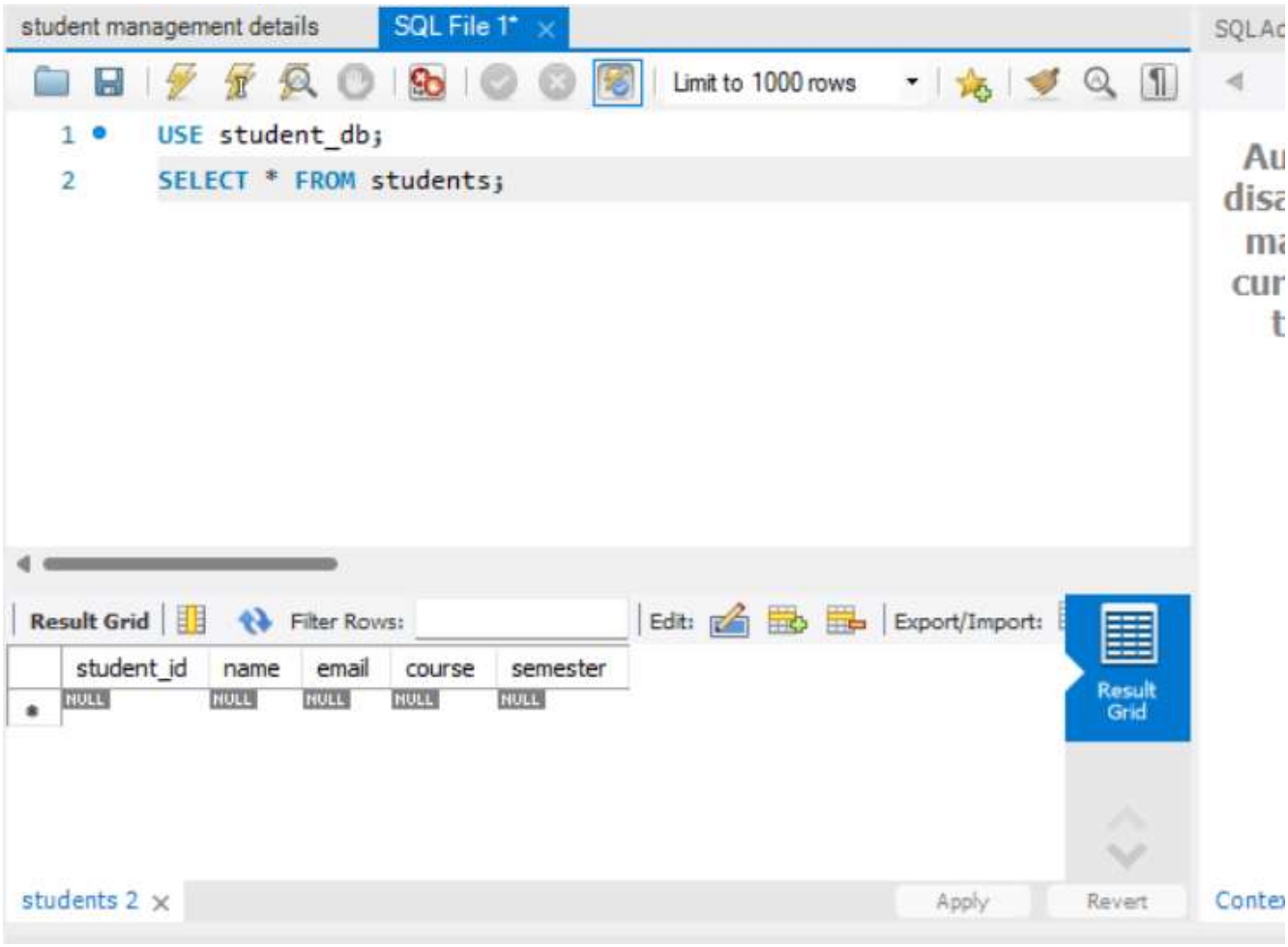


Figure 6. MySQL Workbench result grid after the test record was deleted.

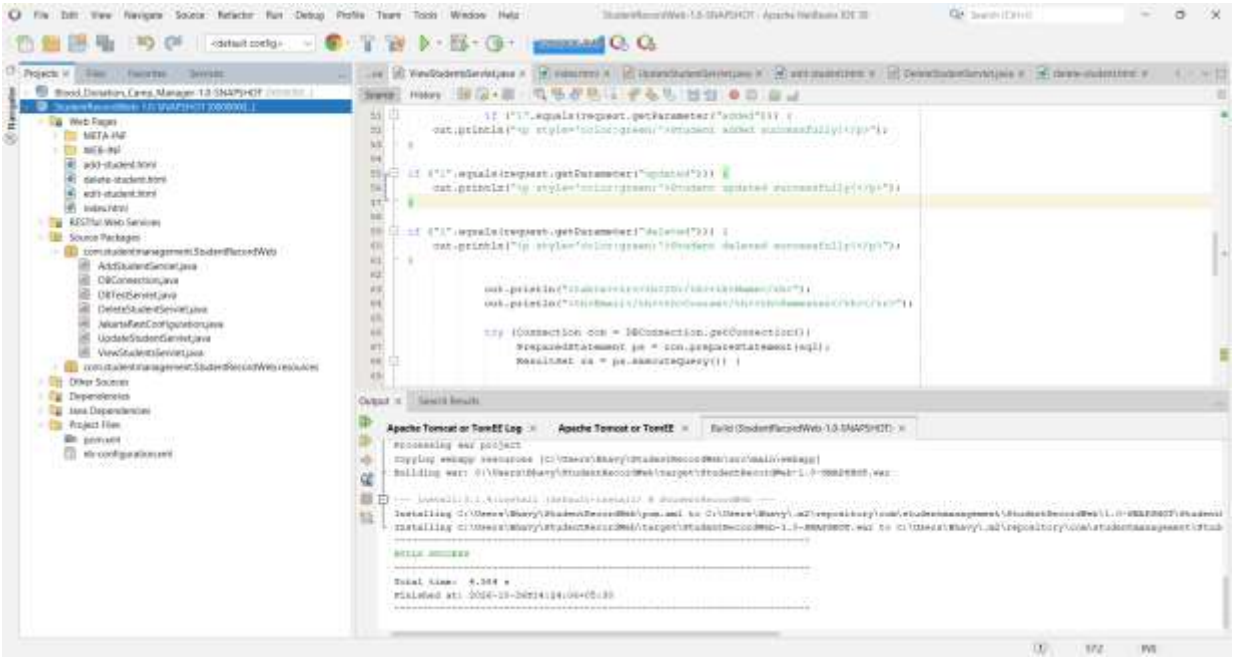


Figure 7. NetBeans project structure showing the web pages and Java servlet classes.

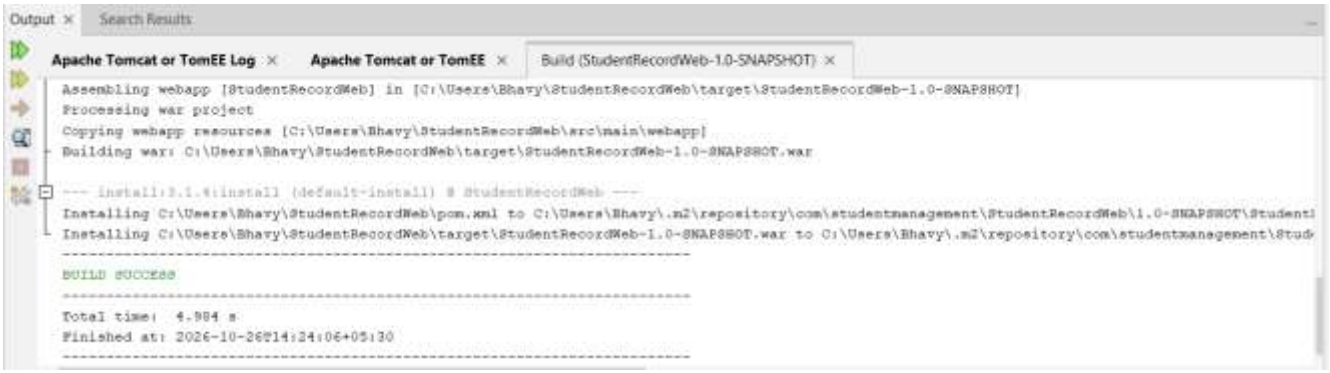


Figure 8. NetBeans output showing BUILD SUCCESS.

8. Testing and Results

Test case	Expected result	Observed result
Add Student	A valid record is saved to MySQL.	Tested successfully.
View Students	Saved records are displayed in a webpage table.	Tested successfully.
Update Student	The selected record is modified and a success message appears.	Tested successfully.
Delete Student	The selected test record is removed and a success message appears.	Tested successfully.
Database verification	SQL query reflects persisted records and deletions.	Verified using MySQL Workbench.
Build and deployment	Project builds and runs on Tomcat.	Build SUCCESS shown in NetBeans.

9. Result

The application successfully demonstrated the main CRUD operations. The website displayed student records, updated a student's semester, and displayed success messages after update and delete operations. Database queries in MySQL Workbench were used to inspect the stored records. NetBeans reported BUILD SUCCESS.

10. Conclusion

The Student Record Management System meets its core objective of managing student information through a Java web application connected to MySQL. The project provided practical experience with HTML forms, Java Servlets, JDBC, SQL queries, and web application deployment. The current version is a basic local application and can be extended with authentication, stronger validation, search and filtering, pagination, and role-based access.

11. Future Enhancements

- Add secure login and user roles.
- Add search, sorting, and pagination for larger student lists.
- Improve validation and user-friendly error messages.
- Add edit/delete actions directly within the student list.

- Deploy to a properly configured server for remote access.

Appendix A: SQL Query Used for Verification

```
USE student_db;
SELECT * FROM students;
```

Appendix B: Project Files

The NetBeans project contains HTML pages for adding, viewing, updating, and deleting student records, Java classes for database connectivity and servlet processing, and a Maven pom.xml file containing project dependencies.

Webpage Link : http://localhost:8080/StudentRecordWeb